DaCaF: Bayes-optimal per-metric inference for probabilistic multi-label classification

Vu-Linh Nguyen[a], **Xuan-Truong Hoang**[b,*], **Anh Hoang**[c], **Van-Nam Huynh**[b]

[a] *Heudiasyc Laboratory, University of Technology of Compiègne, Compiègne, France*

[b] *Japan Advanced Institute of Science and Technology (JAIST), Nomi, Ishikawa, Japan*

[c] *University of Economics Ho Chi Minh City, Ho Chi Minh City, Vietnam*

[*] Corresponding author. E-mail: hxtruong@jaist.ac.jp

Abstract

In multi-label classification (MLC), different evaluation metrics call for different Bayes-Optimal Predictions (BOPs), and no single model is optimal for all of them. Probabilistic MLC allows the target metric to be optimized during the prediction phase, but computing the exact BOP is intractable in general, and existing tools are limited, covering only a few metrics, relying on strong assumptions such as label independence, or lacking a reusable, well-tested implementation. We present DaCaF, a Python package implementing the Divide-and-Conquer and Fusion approach that, given any probabilistic model $P(y \mid x)$, returns the exact BOP for a chosen metric, covering two families of MLC metrics. The package provides seven per-metric prediction rules, each verified against brute-force enumeration, together with a command-line interface, a library API, and reproducible evaluation scripts.

Keywords

Multi-label classification; Probabilistic classifier chains; Bayes-optimal inference; Evaluation metrics

Code metadata

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/hxtruong6/inference_probabilistic_mlc Zenodo: https://doi.org/10.5281/zenodo.20572638 PyPI: https://pypi.org/project/dacaf-mlc/
C3	Permanent link to Reproducible Capsule	https://codeocean.com/capsule/1580907/tree
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python (≥3.10); NumPy, SciPy, scikit-learn, pandas, joblib; optional PyTorch / torchvision / torchxrayvision ([image] extra)
C7	Compilation requirements, operating environments & dependencies	`pip install dacaf-mlc` (core) or `"dacaf-mlc[image]"`; Linux/macOS; Docker image provided
C8	If available Link to developer documentation/manual	README.md, CONVENTIONS.md, docs/
C9	Support email for questions	hxtruong@jaist.ac.jp

1. Motivation and significance

Multi-label classification (MLC) is a setting in which each instance can belong to any subset of the labels, with applications ranging from text and image annotation to high-stakes domains such as clinical decision-making. To compare ground-truth label sets and predictions, a variety of evaluation metrics with different characteristics have been proposed. Probabilistic MLC allows different metrics to be optimized during the prediction phase: once a probabilistic classifier estimates the conditional probability of each labeling, the Bayes-Optimal Prediction (BOP) of a target metric (the prediction $\hat{y}$ that maximizes the expected score of that metric) can be sought. As different metrics may call for significantly different BOPs, a mismatch between the target metric, i.e., the metric optimized during the prediction phase, and the evaluation metric, i.e., the metric used to assess the classifier, can negatively impact predictive performance.

However, computing the BOP in a naive way, by searching over the 2^L candidate predictions for L labels, quickly becomes intractable as the number of labels grows. Existing studies are arguably limited in two respects: some focus on only a small number of metrics, such as subset 0/1 accuracy, Hamming accuracy, and F_β (**???**), yet require a different, metric-specific algorithm for each metric; others rely on strong assumptions, such as label independence (**?**). As a consequence, no reusable, exact, and broadly applicable tool exists for finding BOPs across the metrics commonly used in probabilistic MLC.

We present DaCaF, a software package that implements the Divide-and-Conquer and Fusion (DaCaF) approach (**?**): given any probabilistic model $P(y \mid x)$, it returns the exact BOP for a chosen metric, covering two families of commonly used MLC metrics. It proceeds in two steps, a divide-and-conquer strategy followed by a fusion strategy, to find the prediction that is optimal for the selected metric. DaCaF thereby (a) optimizes the metric one actually cares about, (b) enables studying the mismatch between the target and evaluation metrics in an exact way, undisturbed by approximation, and (c) provides a diagnostic that flags metrics whose BOP has a trivial or very restricted structure, which is useful for assessing the practical value of emerging MLC metrics.

2. Software description

2.1 Software architecture

DaCaF is distributed as the Python package `dacaf_mlc`, which can be installed from PyPI or run through the provided Docker image, and which also exposes a command-line tool `dacaf-mlc`. At its core is the class `ProbabilisticClassifierChainCustom` (PCC), which turns any scikit-learn base estimator into a probabilistic multi-label classifier: it arranges the labels as a classifier chain and, from that chain, estimates the conditional probability $P(y \mid x)$ of each labeling.

Every prediction rule in the package is built from the same two steps of the DaCaF approach. The **divide-and-conquer** step uses a property of Bayes-optimal predictions to partition the 2^L possible predictions into $L+1$ groups according to the number of relevant labels; within each group a local optimum can be found efficiently by sorting the labels by an appropriate score, and the global BOP is then obtained by comparing the local optima across groups. The **fusion** step supplies the probabilistic information that these scores require, namely marginal and pairwise label probabilities, by fusing the predictions of the dependent binary classifiers that make up the chain, rather than by enumerating all labelings.

Around this core, the package provides supporting modules for evaluation metrics and their registry, a pipeline for training and k-fold evaluation, the `evaluate` command-line interface, and loaders for MULAN ARFF datasets and the NIH ChestX-ray data. A trimmed copy of the `skmultiflow` classifier-chain base is vendored so that the package installs and runs on its own.

2.2 Software functionalities

The central functionality is a set of seven per-metric prediction rules (`predict_fmeasure`, `predict_hamming`, `predict_markedness`, `predict_precision`, `predict_npv`, `predict_recall`, and `predict_subset`), each of which returns the prediction that maximizes the expected value of its target metric. Between them these rules cover two families of MLC metrics, so a single fitted model can be queried for the Bayes-optimal prediction of many different metrics. The same machinery also yields a diagnostic that flags metrics whose optimal prediction is trivial or very restricted, which can help practitioners judge the practical value of a newly proposed metric.

For efficiency, the rules share a batched predictor that issues one `predict_proba` call per chain level instead of evaluating all $N \cdot L \cdot 2^L$ labelings, and its output has been verified to match brute-force enumeration numerically on small problems. Finally, the package ships a reproducible evaluation layer (the command-line interface together with scripts over MULAN tabular datasets and the NIH ChestX-ray data) and a correctness harness that checks every rule against exact expected-metric enumeration, run under continuous integration on Python 3.10 and 3.12.

3. Illustrative examples

A few lines are enough to fit a model once and then query the Bayes-optimal prediction for any metric of interest:

```
from sklearn.linear_model import LogisticRegression
from dacaf_mlc.probability_classifier_chains \
    import ProbabilisticClassifierChainCustom
from dacaf_mlc.evaluation_metrics import EvaluationMetrics as EM

pcc = ProbabilisticClassifierChainCustom(
    LogisticRegression(max_iter=10_000))
pcc.fit(X_train, Y_train)            # Y: (n, L) binary
y_f1  = pcc.predict_fmeasure(X_test, beta=1)  # BOP for F1
y_ham = pcc.predict_hamming(X_test)           # BOP for Hamming
print(EM.f_beta(Y_test, y_f1), EM.hamming(Y_test, y_ham))
```

Table **??** reports the headline experiment on the CHD-49 dataset, with PCC and logistic regression averaged over five seeds and ten-fold cross-validation. Each row is the metric that DaCaF optimizes during prediction, and each column the metric used to evaluate the resulting predictions; the bold entry on the diagonal marks the largest value in its column. The pattern is consistent with the divide-and-conquer guarantee: for almost every evaluation metric, the best score in a column is obtained by optimizing that same metric, so a mismatch between

Table 1: CHD-49, PCC + logistic regression, mean over 5 seeds × 10-fold CV (%, higher is better). Read each column: the bold diagonal entry (optimize the metric you evaluate) is the maximum of its column. NPV and Recall collapse to the predict-all trivial optimum.

Target ↓ / Eval →	F_1	Hamming	Markedness	NPV	Precision	Recall	Subset
Predict F_1	**67.07**	69.29	71.91	81.09	62.74	79.22	15.06
Predict Hamming	63.92	**70.78**	71.27	75.74	66.62	66.93	18.05
Predict Markedness	34.20	63.70	**76.96**	71.10	33.37	40.45	8.39
Predict NPV	58.35	43.01	71.50	**100.00**	43.01	99.46	0.00
Predict Precision	40.54	64.83	68.31	63.10	**73.52**	29.10	3.10
Predict Recall	58.35	43.01	71.50	100.00	43.01	**99.46**	0.00
Predict Subset	63.96	69.37	70.35	76.47	64.15	69.79	**18.92**

the target and the evaluation metric measurably lowers performance. The two exceptions are informative rather than accidental. The NPV and Recall columns are both maximized by predictions that collapse to the predict-all labeling (the trivial optimum that the diagnostic flags), which reaches a perfect or near-perfect score without learning anything metric-specific from the data, while driving the Subset score to zero. The whole table is regenerated from the shipped code with a single command, `make reproduce` (CHD-49, the fastest dataset), and the same API carries over unchanged to the NIH ChestX-ray features extracted with a pretrained DenseNet or ResNet backbone.

4. Impact

DaCaF quantifies the exact cost of optimizing a metric other than the one used for evaluation. Because the package is model-agnostic (a drop-in BOP layer on top of any probabilistic MLC model that estimates $P(y \mid x)$), this question can be studied across many estimators and datasets without changing the inference code, and the answer is exact because no label-independence or sampling approximation gets in the way. Coverage is wide: a single package handles two whole families of MLC metrics, and its trivial-optimum diagnostic supports metric selection by flagging metrics whose BOP collapses to a degenerate form, as NPV and Recall do in Table **??**.

The software is built for reuse beyond the companion paper. Every per-metric rule is checked against brute-force enumeration and re-run under continuous integration, so the results can be trusted and rebuilt, and a registry pattern makes adding a new metric or dataset straightforward while the vendored base estimator keeps the install self-contained. Getting started takes little: the package is pip-installable and Dockerized, ships scripted reproduction with bundled MULAN datasets, and lets a newcomer reproduce a paper table in minutes. We expect the main beneficiaries to be MLC researchers studying how evaluation metrics behave, practitioners choosing a metric to optimize for deployment, and educators teaching Bayes-optimal decision-making; an NIH ChestX-ray14 demonstration shows that the same API also works for clinical and image MLC. The permissive MIT license places no restriction on this reuse, so the package can also be integrated into proprietary software and applied in commercial settings.

5. Conclusions

DaCaF turns a proven theoretical recipe into a reusable and well-tested tool that returns the exact Bayes-optimal prediction for a chosen metric, given any probabilistic MLC model. It lets researchers and practitioners optimize the metric they actually care about and study the cost of a mismatch between the target and evaluation metrics without approximation. Future work includes supporting more base models and metric families, and adding optional approximate fast paths for very large label spaces.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Vu-Linh Nguyen: TODO. **Xuan-Truong Hoang**: TODO. **Anh Hoang**: TODO. **Van-Nam Huynh**: TODO.

Acknowledgements
This work was funded/supported by the Junior Professor Chair in Trustworthy AI (Ref. ANR-R311CHD) and University of Economics Ho Chi Minh City (UEH), Vietnam. This work was also sponsored by the Office of Naval Research (ONR) and the Office of Naval Research Global (ONRG) under grant number N62909-23-1-2058. The views and conclusions contained herein are those of the authors only and should not be interpreted as representing those of the U.S. Government.

References

References

V.-L. Nguyen, X.-T. Hoang, A. Hoang, V.-N. Huynh, Probabilistic multi-label classification via a divide-and-conquer and fusion approach, Information Fusion (2026), Article 104517. https://doi.org/10.1016/j.inffus.2026.104517.

X.-T. Hoang, V.-L. Nguyen, A. Hoang, V.-N. Huynh, DaCaF: probabilistic multi-label classification (v1.1.0) [Computer software], Zenodo (2026). https://doi.org/10.5281/zenodo.20572638.

K. Dembczyński, W. Cheng, E. Hüllermeier, Bayes optimal multilabel classification via probabilistic classifier chains, ICML 2010.

K. Dembczyński, W. Waegeman, W. Cheng, E. Hüllermeier, An exact algorithm for F-measure maximization, NeurIPS 2011.

W. Waegeman, K. Dembczyński, A. Jachnik, W. Cheng, E. Hüllermeier, On the Bayes-optimality of F-measure maximizers, J. Mach. Learn. Res. 15 (2014) 3513–3568.

I. Pillai, G. Fumera, F. Roli, Designing multi-label classifiers that maximize F measures: state of the art, Pattern Recognition 61 (2017) 394–404.